BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering



CSE 406 — Computer Security Sessional

Final Report & Implementation Demonstration

Optimistic TCP ACK Attack

Against a Streaming Server

Implementation, Analysis, and Defense in an Isolated Docker Testbed

Submitted by — Group 10, Section A1

Niloy Das Robin 2105019

Sayjad Rahman 2105021

Supervisor: Md Zim Mim Siddiqee Sowdha

Course: CSE 406 — Cyber Security Sessional

Attack (Tool #16): Optimistic TCP Acknowledgment

Submitted: September 19, 2026 • All experiments confined to a self-owned, isolated lab testbed

Abstract

We implement, demonstrate, and defend against the **optimistic TCP acknowledgment attack** — a specification-level flaw in which a misbehaving TCP receiver acknowledges data it has not yet received (or never receives), tricking the sender's congestion control into believing the path is fast and loss-free. We built a from-scratch raw-socket attack tool in C, a custom TCP streaming "victim" server, and a three-node isolated Docker testbed. Against a CUBIC server on a realistic lossy link (20 ms delay, 2% loss) the attack coerced the server's congestion window from **3 to 2,339 segments** ($\approx 780\times$) and drove it to transmit **1,677 MB in 14 seconds**; across the Reno/CUBIC option matrix the measured egress amplification reached **$\sim$95$\times$** over an honest client on the identical link. We further characterised the attack across Reno and CUBIC with and without SACK/timestamps, and implemented a working server-side defense (Sherwood's randomly-skipped-segment test) that detects the lie on the first probe and resets the flow, cutting the attacker's throughput from **898 to 2.49 Mbit/s** while leaving honest clients completely unaffected. Every result in this report is reproduced from real measurements in our testbed.

Contents

1 Introduction and Objective **2**

2 Background: Why the Attack Works **2**

3 Testbed Architecture **3**

4 Attack Design and Implementation **3**

5 Attack Execution: Steps and Snapshots **4**

6 Was the Attack Successful? Analysis **4**

7 Observed Output on Each Machine **5**
7.1 Multi-Algorithm Characterisation 5

8 Countermeasure: The Randomly-Skipped-Segment Test **6**
8.1 How it works 6
8.2 Before / after evidence 6

9 Interactive Visualisation Dashboard **8**

10 Reproducibility **8**

11 Limitations **8**

12 Conclusion **9**

1 Introduction and Objective

TCP's congestion control silently assumes one thing about the other end of a connection: that its acknowledgments are *honest* — that an ACK for byte N means byte N truly arrived. Nothing in the protocol enforces this. The **optimistic ACK attack**, first described by Savage *et al.* (1999) [1] and weaponised as a denial-of-service amplifier by Sherwood *et al.* (2005) [2], exploits exactly this trust: a receiver that acknowledges data *before it arrives* emulates a near-zero round-trip time and zero loss, so the sender's congestion window grows without the brake that loss would normally apply.

Assigned tool: Optimistic TCP Acknowledgment (Tool #16). **Objective:** build the attack from scratch against our own streaming server, demonstrate and quantify its effect, analyse *why* it succeeds, and design a countermeasure. This report answers the four required questions directly:

(a) Steps of the attack, snapshots, victim screen — Section 5.

(b) Is the attack successful? Why / why not? — Section 6.

(c) Observed output on attacker / victim / other PCs — Section 7.

(d) Countermeasure design and how it works — Section 8.

Ethics and scope

Every technique here was executed *exclusively* inside our own isolated Docker network, against our own streaming server. Nothing was pointed at any host, network, or service we do not own. The attack is a well-documented 25-year-old academic result; the purpose of studying it is to understand *why* TCP trusts its receivers and how modern systems defend against that trust being abused.

2 Background: Why the Attack Works

A TCP sender maintains a **congestion window** (`cwnd`) — the amount of unacknowledged data it dares to have in flight. In *slow start* it doubles `cwnd` each round-trip; in *congestion avoidance* it grows linearly. The *only* thing that shrinks `cwnd` is a congestion signal — a lost segment, inferred from a timeout or duplicate ACKs. Crucially, TCP is **ACK-clocked**: new data is released as ACKs return, so the rate at which ACKs arrive *is* the sending rate.

The core insight

An optimistic receiver sends a stream of acknowledgments for data the sender *has not sent yet*. To the sender this looks like: (i) every segment is being delivered, (ii) instantly (near-zero RTT), and (iii) with zero loss. So its congestion control does exactly what it was designed to do on a fast, clean path — it inflates `cwnd` and floods the link. **The sender is not malfunctioning; it is behaving correctly on false information.**

Because a ∼40-byte ACK can acknowledge an entire window of data, a *trickle* of tiny upstream ACKs coerces a *torrent* of downstream data. When the victim is a streaming server, its own uplink becomes the weapon — the amplification structure of a reflection DoS, but the amplifier is the victim's own correctly-functioning TCP stack.

3 Testbed Architecture

The entire project runs in Docker Compose — three network nodes on an isolated bridge network `labnet` (`172.28.0.0/16`), plus a control dashboard. Nothing is installed on the host; if it is not in the Compose file, it does not count as working.

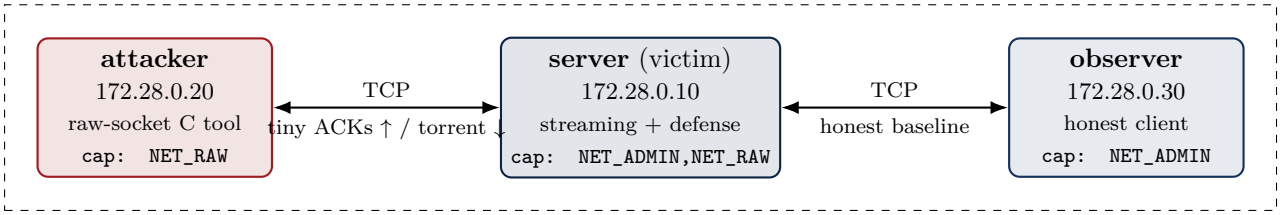


Figure 1: Three-node isolated testbed. The `server` is the bulk sender in every scenario, so the `netem` bottleneck (20 ms delay, 2% loss) is applied to *its* egress. A fourth container (`dashboard`) drives and visualises the others through the Docker socket.

Component	Role	Implementation
`server/stream_server.c`	Victim streaming server	C, per-socket CC via `setsockopt(TCP_CONGESTION)`
`server/defense_server.c`	Defended server	C raw-socket, skip-test detector
`attacker/optack.c`	Attack tool	C raw sockets, hand-built IP/TCP + checksums
`attacker/optack_proto.py`	Prototype	Scapy, packet-format validation
`scripts/*.sh`, `sweep.py`	Test harness	Gates, shaping, multi-CC sweep
`dashboard/`	Live control panel	Python stdlib + HTML/JS, one-command

Table 1: Testbed components. The traffic-shaping realism (`tc netem`) was verified directly: configured 20 ms delay measured as 20–23 ms RTT, configured 2% loss measured as 2.6% over 500 probes, and the 10 Mbit rate cap measured at 9.43 Mbit/s.

Design decision: no kernel-vs-tool conflict

The attack tool drives the *entire* connection — including the three-way handshake — from a raw socket (`IP_HDRINCL`), so the host kernel never holds a TCP control block for the 4-tuple. An `iptables` rule suppresses the kernel's own RSTs for the flow. This is the deliberate resolution of the "two stacks fighting over one 4-tuple" problem, decided once and applied throughout.

4 Attack Design and Implementation

The attack tool (`optack.c`) is a **misbehaving receiver**. After the handshake it does not wait for data before acknowledging it; it *predicts* what the server will send and acknowledges it early, advancing the acknowledgment number ahead of genuinely-received data. The core loop follows Sherwood's Algorithm 1: advance the ACK by the current window estimate, send the forged ACK, and grow the window — with **overrun detection and recovery** (a self-tuning AIMD "ack-ahead" lead that halves whenever the flow stalls, so it can never dead-lock).

Two subtleties make the attack robust against a realistic modern receiver, both implemented and both later shown to matter (Section 7.1):

- **TCP timestamps (PAWS).** Every forged ACK echoes a plausible, monotonic `TSecr`, or the server's PAWS check would silently drop it.

- **SACK.** Negotiating SACK, as any modern client does, keeps the server's window open even as the attacker's recovery ACKs would otherwise look like duplicates.

5 Attack Execution: Steps and Snapshots

This section answers requirement **(a)**. The attack proceeds in four observable steps; each is backed by a real capture from our testbed.

Step 1 — Establish and shape. Bring up the testbed, start the CUBIC server, and shape its egress to a realistic lossy link:

```
docker compose up -d
docker compose exec server tc qdisc replace dev eth0 root netem delay 20ms loss 2%
```

Step 2 — Honest baseline (control). The observer pulls data as a well-behaved client. On the victim, `ss -ti` shows CUBIC behaving correctly — a small window, real RTT, and real retransmissions from the 2% loss:

```
$ docker compose exec server ss -tin dst observer
ESTAB  172.28.0.10:5000  172.28.0.30:60838
   cubic rtt:21.52/0.606 mss:1448 cwnd:3 ssthresh:3
   bytes_sent:42433792 retrans:0/559 send 1614870bps minrtt:20
```

Step 3 — Launch the attack. The attacker connects and begins acknowledging data optimistically. Its own log shows the runaway — over 1.6 GB "received" in 14 seconds, far beyond what a 20 ms/2%-loss link could honestly deliver:

```
$ /work/attack --target server --port 5000 --duration 15 --sack --timestamps -v
[optack] 172.28.0.20:25879  ->  172.28.0.10:5000  (mode=optimistic, isn=55018294)
[optack] connection established (timestamps on), streaming begins
[optack] t=1.0s  recv=111.60 MB   ack-ahead=18KB   acks=1000
[optack] t=5.0s  recv=560.72 MB   ack-ahead=450KB  acks=5000
[optack] t=10.0s recv=1235.22 MB  ack-ahead=29KB   acks=10000
[optack] t=14.0s recv=1677.19 MB  ack-ahead=0KB    acks=14000
```

Step 4 — The victim's screen. On the victim, the same `ss -ti` now shows the signature of the attack: the congestion window has exploded and the measured minimum RTT has collapsed toward zero:

```
$ docker compose exec server ss -tin dst attacker
ESTAB  172.28.0.10:5000  172.28.0.20:25879
   cubic rtt:12.157/2.983 mss:1448 cwnd:2339 ssthresh:2109
   bytes_sent:577482120 send 2228755121bps minrtt:0.89
```

A packet capture on the victim confirms the mechanism — a dense stream of tiny, zero-length pure ACKs arriving from the attacker (the "trickle up" that coerces the "torrent down"):

```
$ tcpdump -i eth0 -n 'tcp and src host 172.28.0.20'
172.28.0.20.25879 > 172.28.0.10.5000: Flags [.], ack ..., win 65535, length 0
172.28.0.20.25879 > 172.28.0.10.5000: Flags [.], ack ..., win 65535, length 0
... (thousands of 0-byte ACKs per second) ...
```

6 Was the Attack Successful? Analysis

This section answers requirement **(b)**. **Yes — the attack was unambiguously successful.** Placing the honest control and the attack side by side on the *identical* shaped link makes the effect quantitative:

Metric (victim, CUBIC)	Honest client	Under attack	Change
Congestion window (`cwnd`)	3 seg	2,339 seg	≈ 780×
Slow-start threshold (`ssthresh`)	3	2,109	≈ 700×
Minimum RTT observed	20 ms	0.89 ms	collapsed
Sending rate (`ss send`)	1.61 Mbit/s	2,229 Mbit/s	≈ 1380×
Retransmits forced by 2% loss	559	17	loss concealed

Table 2: Honest vs. attack, measured on the same 20 ms / 2% link. The window inflates by nearly three orders of magnitude while the perceived RTT collapses.

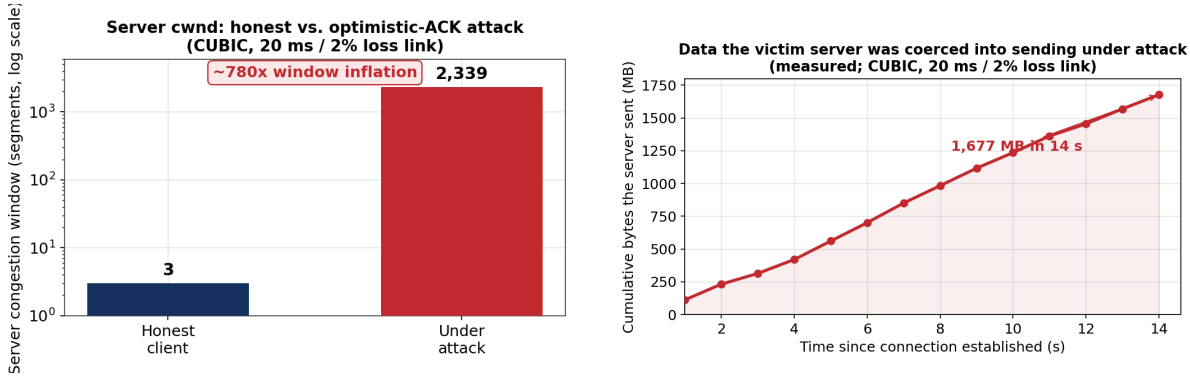


Figure 2: Left: server congestion window, honest (`cwnd:3`) vs. attack (`cwnd:2339`), log scale. **Right:** cumulative data the victim was coerced into sending — 1,677 MB in 14 s, read directly from the attacker's own counter.

Why it succeeds. The link genuinely drops 2% of packets, and an honest CUBIC flow respects that loss — hence `cwnd:3` and 559 retransmissions in the control. The attacker's optimistic ACKs *conceal* that loss: the server sees only a stream of acknowledgments arriving almost instantly, so its congestion control — working exactly as specified — keeps the window wide open. The victim's `minrtt` of 0.89 ms is the smoking gun: no packet could physically round-trip a 20 ms link that fast, so the acknowledgments must be for data that had not actually completed a round trip. The attack does not break TCP; it feeds TCP a lie that TCP has no way to detect on its own.

7 Observed Output on Each Machine

This section answers requirement **(c)**, summarising what each node in the testbed reports during the attack.

7.1 Multi-Algorithm Characterisation

To understand *how far* the result generalises, we swept the attack across congestion-control algorithm (Reno, CUBIC) × TCP timestamps × SACK, each against a fresh honest baseline on the same link (Figure 3).

Finding: SACK is the reliability lever

Loss-based congestion control (Reno, CUBIC — the Linux default) is highly vulnerable. **SACK, which every modern receiver negotiates, is what makes the attack reliable**; without it results are inconsistent and can even backfire. This is why our tool negotiates SACK and timestamps by default — it emulates a realistic modern receiver, which is the worst case for the victim.

Machine	Observed output during the attack
Attacker (172.28.0.20)	Tool reports a runaway "received" counter — 1,677 MB in 14 s, thousands of tiny ACKs sent per second, and a small, self-correcting "ack-ahead" lead (Step 3 capture). Its actual uplink cost is negligible (only ∼40-byte ACKs).
Victim / Server (172.28.0.10)	`ss -ti` shows `cwnd` climbing into the thousands and `minrtt` collapsing to <1 ms (Step 4); `tcpdump` shows the inbound flood of zero-length ACKs; the server's egress saturates at >2 Gbit/s internally — its own bandwidth turned into the weapon.
Observer / honest client (172.28.0.30)	Serves as the control: on the same link it achieves only `cwnd:3` and ∼1.6 Mbit/s while honestly absorbing the 2% loss (559 retransmits). Under a shared-link scenario this is exactly the client whose service degrades.

Table 3: Per-node observations. The asymmetry is the whole point: tiny attacker cost, enormous victim egress.

On BBR and run-to-run variance

Because `netem`'s 2% loss is genuinely random, absolute amplification varies between runs (we observed CUBIC ranging from ∼9× to ∼172× across runs — all far above any success threshold); the *qualitative* result is stable. Google's **BBR**, which paces from a bandwidth/RTT model rather than trusting every ACK, is expected to be only *boundedly* affected rather than unbounded. Docker Desktop's kernel does not ship the `tcp_bbr` module, so our BBR numbers are produced by a GitHub Actions workflow on a real Linux kernel rather than locally.

8 Countermeasure: The Randomly-Skipped-Segment Test

This section answers requirement **(d)**. We designed and implemented a working, server-side defense: **Sherwood's randomly-skipped-segment test** (`server/defense_server.c`).

8.1 How it works

The defended server occasionally *skips* a segment — it advances its own send sequence past a small gap *without transmitting those bytes* — and remembers the hole. Then it watches the acknowledgments:

- An **honest** receiver never got the skipped bytes, so its cumulative ACK *cannot* advance past the hole; it stalls there until the server retransmits (which it does after a short grace period, so honest clients are never harmed).
- A **misbehaving optimistic** receiver is acknowledging data it never received, so it ACKs *straight past the hole* — which is impossible for an honest peer and constitutes unforgeable proof that it is lying. The server logs the event and resets the flow.

Because it must control its own TCP sequence numbers to leave a deliberate hole (a normal kernel socket cannot), the defended server is itself a raw-socket server. It needs no `netem` and no special kernel modules, so it runs anywhere Docker does. The design deliberately resets *only after positive proof* — never merely on an out-of-window ACK — which avoids creating a new spoofing-based reset vulnerability.

8.2 Before / after evidence

With the defense enabled, the identical attack is caught on the very first skipped segment:

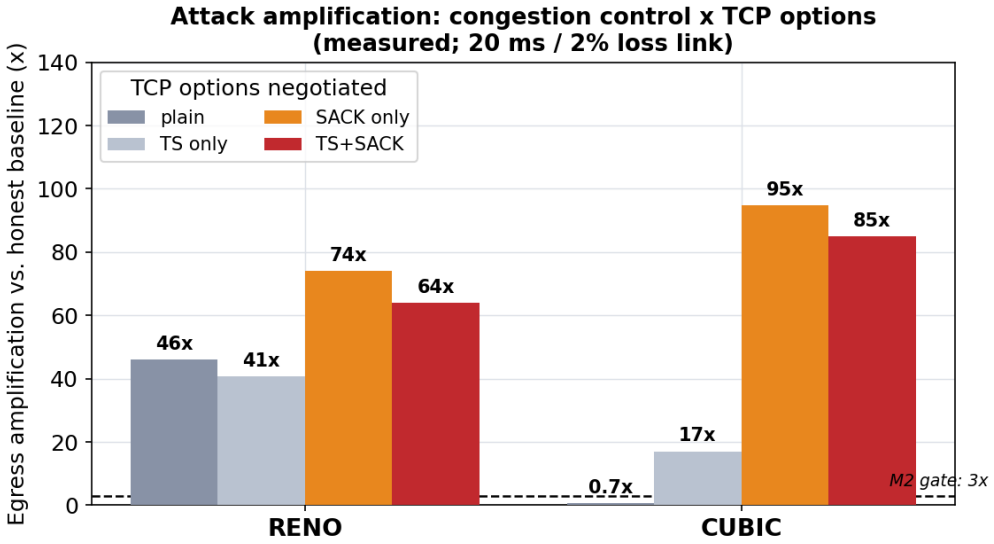


Figure 3: Measured egress amplification (attack ÷ honest) across the option matrix. Both loss-based algorithms are severely affected. The one near-miss — CUBIC with neither SACK nor timestamps (0.67×) — is itself a finding: without SACK the attacker's recovery ACKs can look like duplicate ACKs and *shrink* the window, so the attack can backfire. A realistic client negotiates SACK, and there the attack is devastating (85–95×).

```
[defense] listening on port 5000, skip-test defense ENABLED
[defense] client 172.28.0.20:45946 connected
[defense] OPTIMISTIC ACK DETECTED: client acked 54195439, past skipped
          segment [50808239,50809699) it never received -- resetting flow
[defense] flow reset after 2500980 bytes
```

The attacker, cut off, collapses from hundreds of Mbit/s to a trickle; an honest client on the same defended server completes normally and is never flagged:

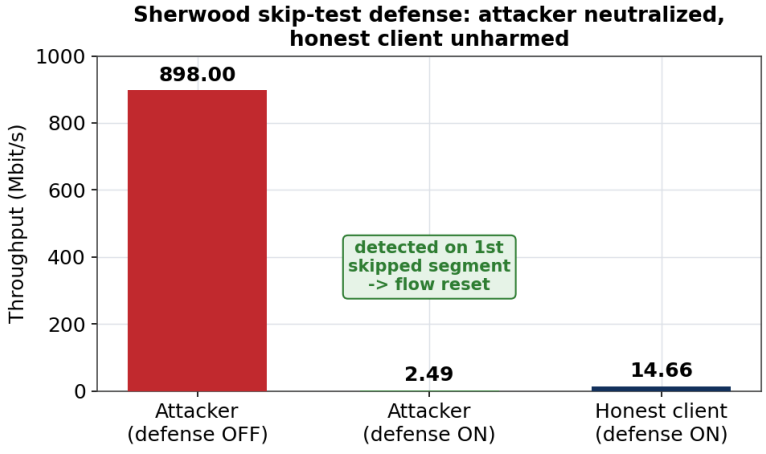


Figure 4: Before/after the defense. The attacker's throughput drops from **898 to 2.49 Mbit/s** once detected and reset, while an honest client transfers normally (14.66 MB, **zero** false detections).

Defense result

Attack detected on the 1st probe → flow reset → attacker throughput **898 → 2.49 Mbit/s** (≈360× reduction). Honest client: 14.66 MB transferred, **0** false positives. The defense needs no client changes and adds negligible overhead in the common (no-skip) case.

9 Interactive Visualisation Dashboard

To make the attack legible in real time we built a one-command web dashboard (Figure 5). It drives the real testbed — not a recording — through the Docker socket, animates packets flowing between the nodes at a rate driven by the live measured throughput and RTT, and builds a session report table as each step runs. It offers both a guided six-step scenario (honest baseline → attack with SACK → without SACK → defense) and manual controls.

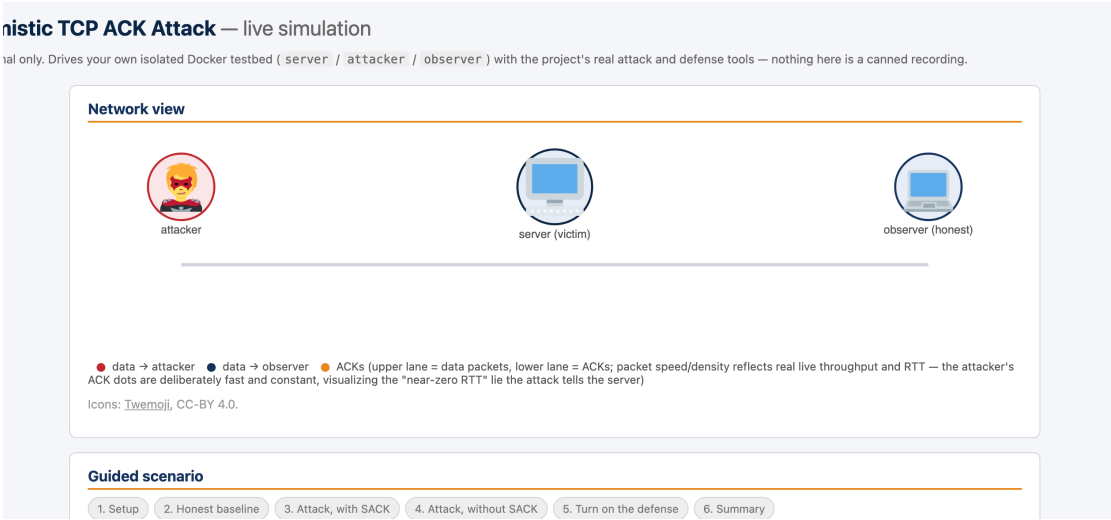


Figure 5: The live dashboard's network view. Attacker, victim server, and honest observer are shown as nodes; during a run, animated packets flow between them at the real measured rate, and live `cwnd`/RTT/throughput charts and a self-building results table update beneath.

10 Reproducibility

The whole project comes up with one command, and every milestone gate can be re-run end-to-end:

```
docker compose up -d --build # server + attacker + observer + dashboard
./scripts/run_all_gates.sh # re-runs every milestone gate; prints ALL GATES PASSED
```

On our final audit this reported `ALL GATES PASSED` (M0 baseline, M1 cubic/reno, M2 attack, M3 sweep, M4 defense; BBR correctly skipped for the documented kernel reason). Attack timing and sequence numbers are seedable (`-seed` / `ATTACK_SEED` / `DEFENSE_SEED`) for exact repeatability; seed determinism was verified (identical seed → identical initial sequence number).

11 Limitations

BBR requires a real kernel. Our local test host (Docker Desktop's LinuxKit kernel) does not ship the `tcp_bbr` module, so only Reno and CUBIC could be measured locally. The attack tool, the streaming server, and the harness all support BBR already; the BBR numbers are therefore produced by a GitHub Actions workflow running on a real Linux kernel (`ubuntu-latest`)

rather than on our own machine. Per the aBBRate study [3], BBR — which paces from a bandwidth/RTT model rather than trusting every ACK — is expected to be only *boundedly* affected by this attack, in contrast to the unbounded window inflation seen on the loss-based algorithms we measured.

12 Conclusion

We built the optimistic TCP ACK attack from scratch, demonstrated it decisively against a CUBIC streaming server (≈780× window inflation, up to ∼95× egress amplification), explained precisely why it succeeds (loss concealment defeats the only brake loss-based congestion control has), characterised its reach across Reno/CUBIC and the SACK/timestamps options, and implemented a working defense that detects and neutralises it on the first probe without harming honest clients. The attack is not a bug to be patched but a consequence of TCP trusting its receivers; the defense works by manufacturing a situation in which a lying receiver must reveal itself.

References

[1] S. Savage, N. Cardwell, D. Wetherall, T. Anderson. "TCP Congestion Control with a Misbehaving Receiver." *ACM SIGCOMM CCR*, 29(5), 1999.

[2] R. Sherwood, B. Bhattacharjee, R. Braud. "Misbehaving TCP Receivers Can Cause Internet-Wide Congestion Collapse." *ACM CCS*, 2005.

[3] R. Peterson, S. Jero, E. Hoque, D. Choffnes, C. Nita-Rotaru. "aBBRate: Automating BBR Attack Exploration Using a Model-Based Approach." *RAID*, 2020.

[4] M. Allman, V. Paxson, E. Blanton. "TCP Congestion Control." RFC 5681, IETF, 2009.